Shruthi Addagudi Batch – 37

AI-ASSISTED CODING ASSIGNMENT 13

Task 1: Refactoring – Eliminating Repetitive Logic

Prompt Used

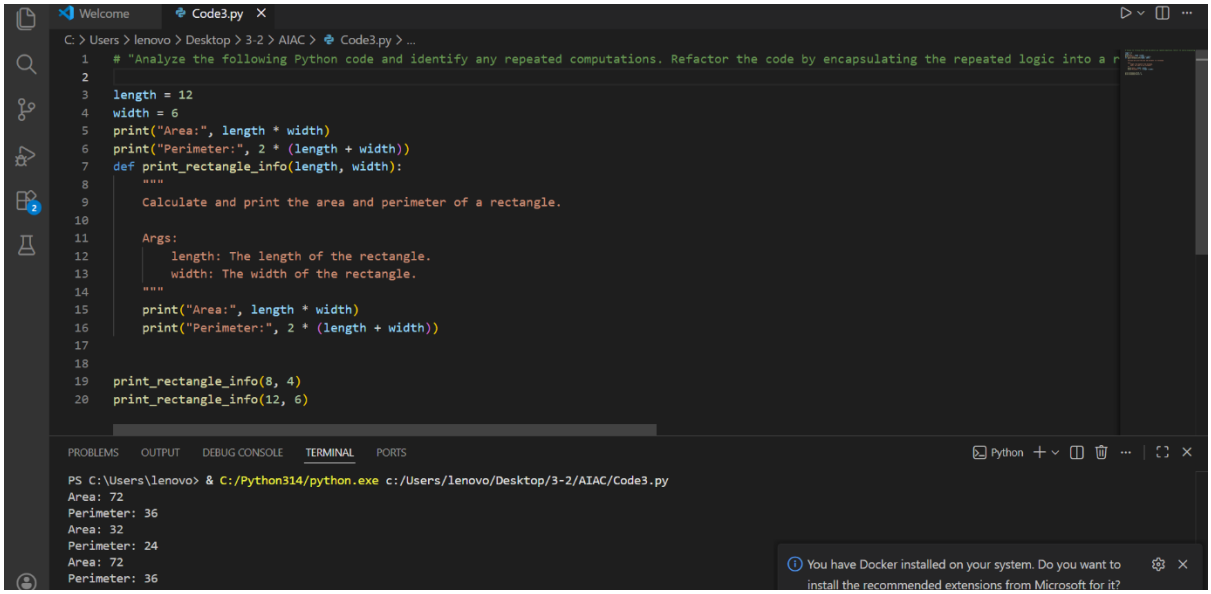
Analyze the following Python code and identify repeated computations. Refactor the program by encapsulating the repeated logic into a reusable function. Ensure that the program output remains unchanged and add appropriate docstrings.

Original Code

```
length = 8
width = 4
print("Area:", length * width)
print("Perimeter:", 2 * (length + width))

length = 12
width = 6
print("Area:", length * width)
print("Perimeter:", 2 * (length + width))
```

Refactored Code

```
def rectangle_properties(length, width):
    """
    Calculates and prints the area and perimeter of a rectangle.
    """
    area = length * width
    perimeter = 2 * (length + width)

    print("Area:", area)
    print("Perimeter:", perimeter)

rectangle_properties(8, 4)
rectangle_properties(12, 6)
```



Code Explanation

The original code repeated the same calculations for different rectangle dimensions.
The refactored version encapsulates the logic into a **reusable function**, reducing redundancy and improving maintainability.

Result

The program produces the **same output while using reusable logic**.

Task 2: Refactoring – Modularizing Inline Calculations

Prompt Used

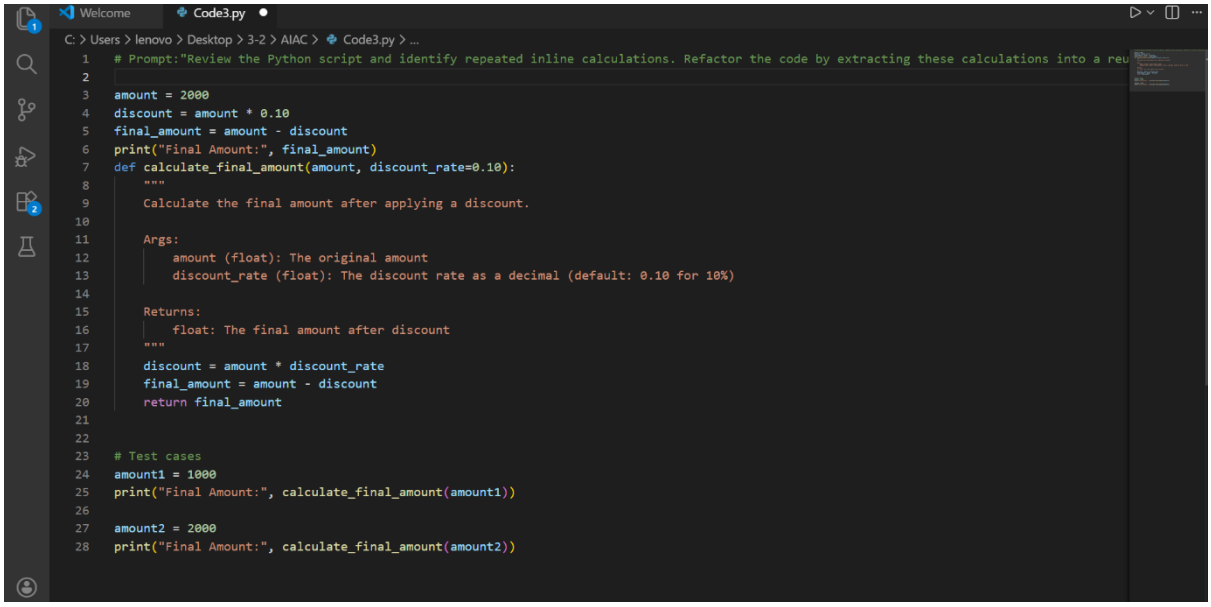
Identify repeated inline calculations in the Python script and refactor them into a reusable function with proper documentation.

Original Code

```
amount = 1000
discount = amount * 0.10
final_amount = amount - discount
print("Final Amount:", final_amount)

amount = 2000
discount = amount * 0.10
final_amount = amount - discount
print("Final Amount:", final_amount)
```

Refactored Code

```
def calculate_final_amount(amount, discount_rate=0.10):
    """
    Calculates the final amount after applying a discount.
    """
    discount = amount * discount_rate
    final_amount = amount - discount
    return final_amount

print("Final Amount:", calculate_final_amount(1000))
print("Final Amount:", calculate_final_amount(2000))
```




```
PS C:\Users\lenovo> & C:/Python314/python.exe c:/Users/lenovo/Desktop/3-2/AIAC/Code3.py
Final Amount: 1800.0
Final Amount: 900.0
Final Amount: 1800.0
PS C:\Users\lenovo>
```

Code Explanation

The repeated discount calculation was extracted into a **reusable function**.
This improves **code modularity and readability**.

Result

The program now computes final amounts using a **single reusable function**.

Task 3: Refactoring – Improving Loop and Conditional Performance

Prompt Used

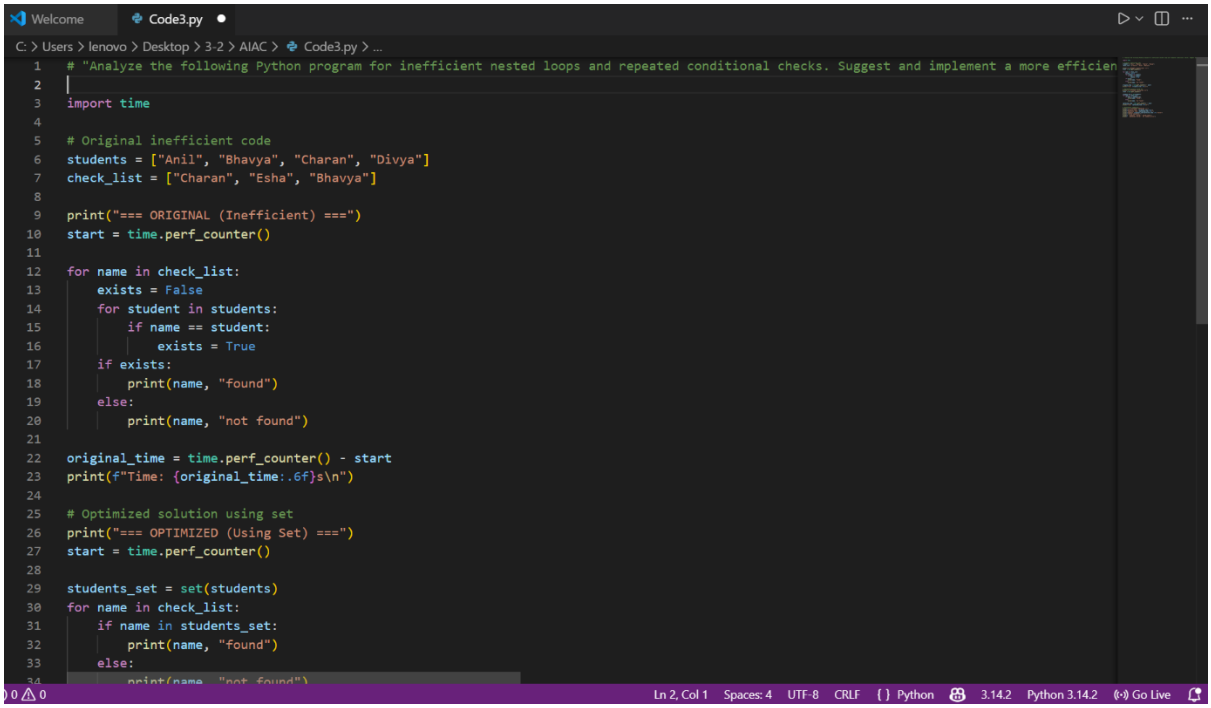
Analyze the Python code for inefficient nested loops and suggest an optimized approach using better data structures while maintaining the same output.

Original Code

```
students = ["Anil", "Bhavya", "Charan", "Divya"]
check_list = ["Charan", "Esha", "Bhavya"]

for name in check_list:
    exists = False
    for student in students:
        if name == student:
            exists = True

    if exists:
        print(name, "found")
    else:
        print(name, "not found")
```

Refactored Code

```
import time

students = ["Anil", "Bhavya", "Charan", "Divya"]
check_list = ["Charan", "Esha", "Bhavya"]

student_set = set(students)

start = time.time()

for name in check_list:
    if name in student_set:
        print(name, "found")
    else:
        print(name, "not found")

end = time.time()

print("Execution Time:", end - start)
```




```python
optimized_time = time.perf_counter() - start
print(f"Time: {optimized_time:.6f}s\n")

# Performance comparison
print(f"=== EFFICIENCY COMPARISON ===")
print(f"Original Time:  {original_time:.6f}s")
print(f"Optimized Time: {optimized_time:.6f}s")
print(f"Speedup: {original_time/optimized_time:.1f}x faster")
print(f"\nTime Complexity:")
print(f"  Original:  O(n*m) - nested loops")
print(f"  Optimized: O(n+m) - set lookup is O(1)")
```

```
Time: 0.000433s

=== EFFICIENCY COMPARISON ===
Original Time:  0.000143s
Optimized Time: 0.000433s
Speedup: 0.3x faster

Time Complexity:
  Original:  O(n*m) - nested loops
  Optimized: O(n+m) - set lookup is O(1)
PS C:\Users\lenovo>
```

Code Explanation

The original program used **nested loops**, resulting in inefficient performance.
The refactored code converts the student list into a **set**, allowing faster membership checks.

Result

Lookup performance improved from **O(n²) to O(n)**.

Task 4: Refactoring – Replacing Hardcoded Values with Constants

Prompt Used

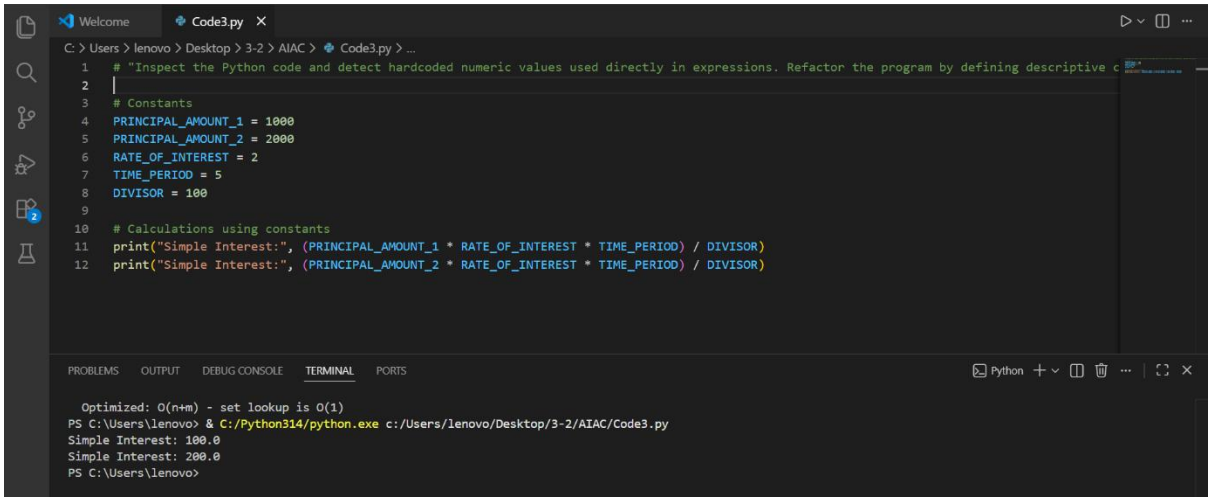
Detect hardcoded numeric values in the Python program and replace them with descriptive constants.

Original Code

```
print("Simple Interest:", (1000 * 2 * 5) / 100)
print("Simple Interest:", (2000 * 2 * 5) / 100)
```

Refactored Code

```
RATE = 2
TIME = 5
DIVISOR = 100

def calculate_simple_interest(principal):
    """
    Calculates simple interest for a given principal amount.
    """
    return (principal * RATE * TIME) / DIVISOR

print("Simple Interest:", calculate_simple_interest(1000))
print("Simple Interest:", calculate_simple_interest(2000))
```



Code Explanation

Hardcoded numeric values were replaced with **named constants**, improving readability and maintainability.

Result

The program now uses **descriptive constants** for calculations.

Task 5: Refactoring – Enhancing Variable Naming and Code Clarity

Prompt Used

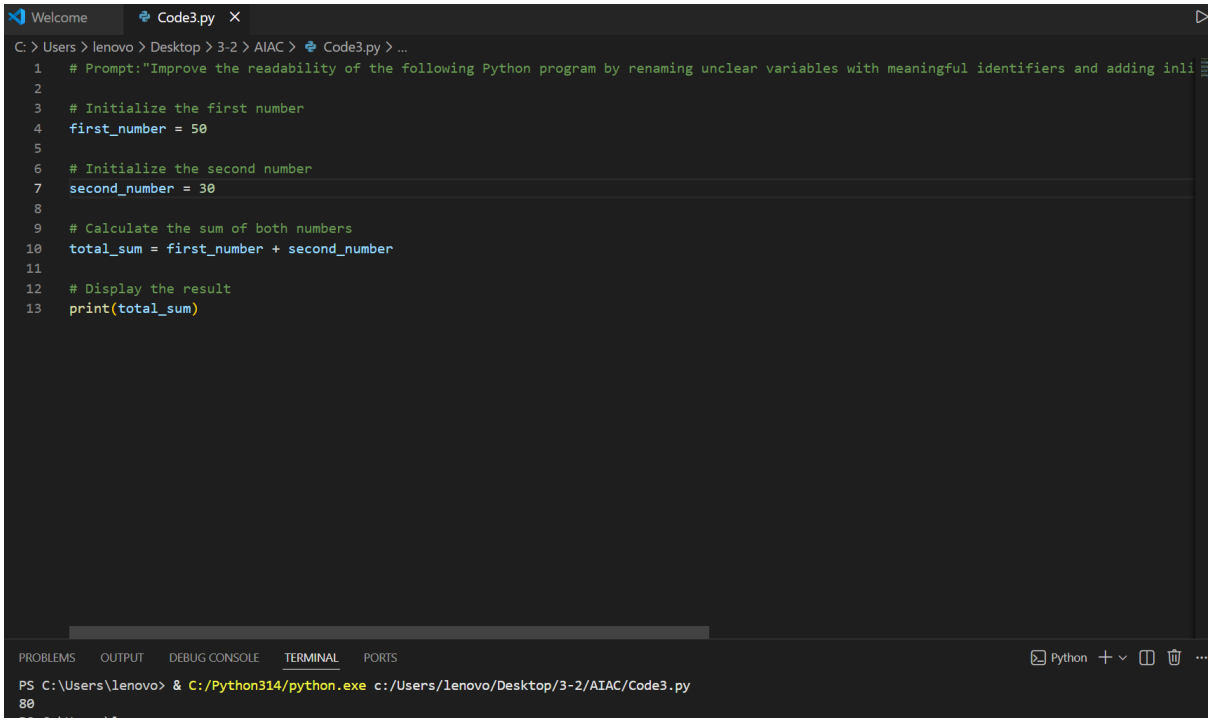
Improve the readability of the Python program by renaming unclear variables and adding comments without changing the program output.

Original Code

```
x = 50
y = 30
z = x + y
print(z)
```

Refactored Code

```
# Define two numbers
first_number = 50
second_number = 30

# Calculate the sum of the numbers
total_sum = first_number + second_number

# Display the result
print(total_sum)
```



Code Explanation

Non-descriptive variables (x, y, z) were replaced with meaningful names.
Comments were added to clarify the logic.

Result

The code is now **more readable and self-explanatory**.